

1.1 A Brief Introduction to Recursion :

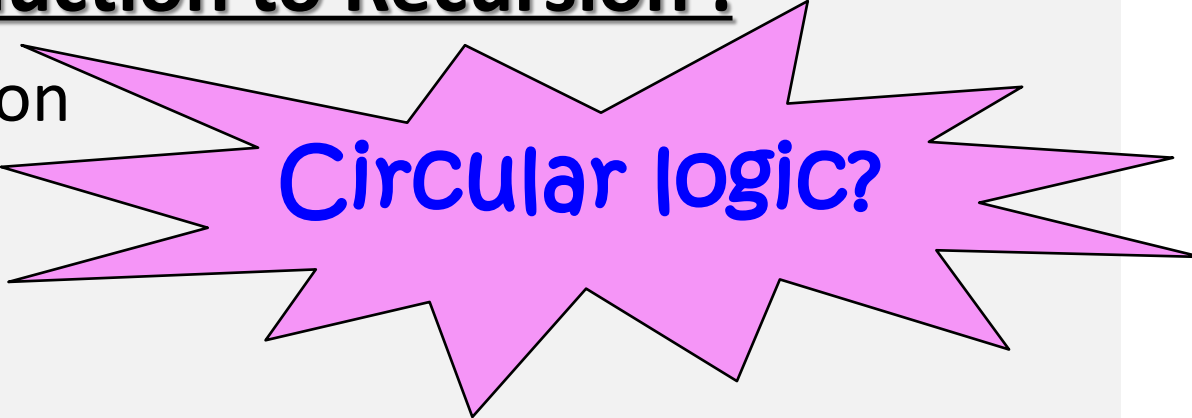
Mathematical function

1. $C = 2(F - 32) / 9$

2. $y = \sin(x) * \pi$

3. $f(x) = 2f(x-1) + x^2$

$f(0) = 0$, x nonnegative integer



Circular logic?



4. Factorial กำหนด x เป็นจำนวนเต็มที่ไม่เป็นลบ (non-negative integer)

$$f(x) = x * f(x-1) \quad , \quad f(1) = 1 \quad \text{และ} \quad f(0)=1$$

5. Fibonacci number กำหนด x คือ non-negative integer
Fibonacci number คือลำดับของจำนวนเต็มที่จำนวนถัดไปเกิดจากการเอา 2 จำนวนก่อนหน้ามารวมกัน

$$f(n) = f(n-1) + f(n-2)$$

$$f(0)=0 \quad , \quad f(1) = 1$$



Example 1

```
#include <iostream>
using namespace std;
int main()
{   int ans;
    ans = fact(3);
    cout << ans;
}
```

```
int fact(int x)
{   if(x <= 1)
        return 1;
    else
        return x* fact(x-1);
}
```



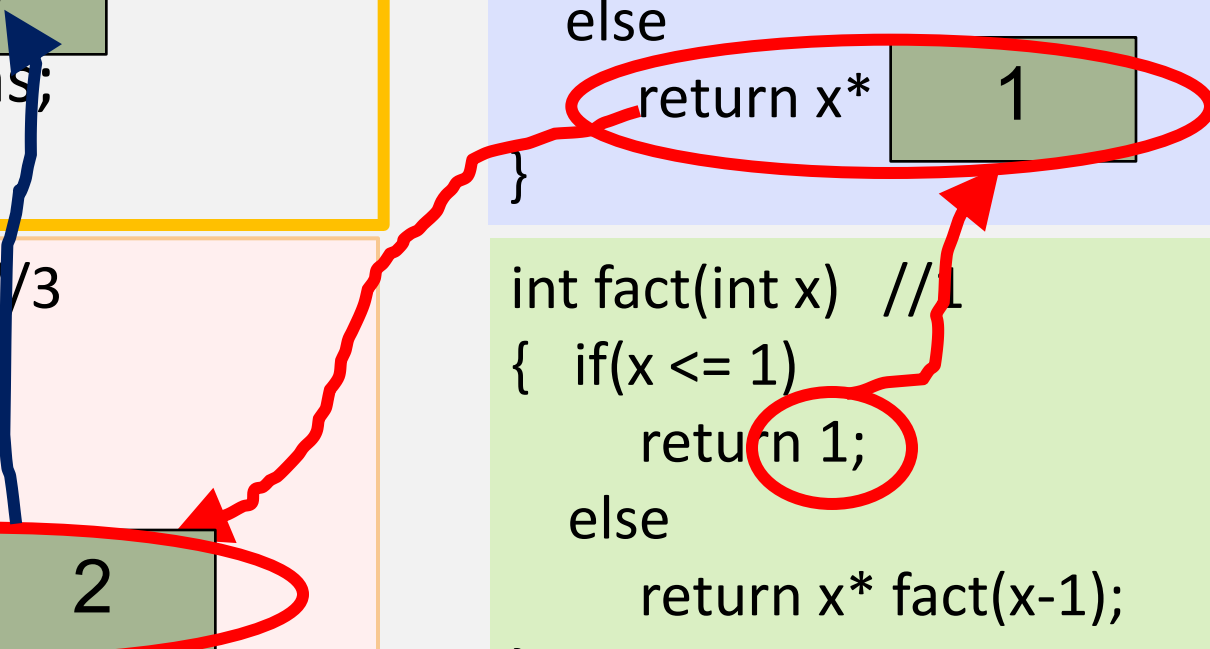
Example 2 การทำงานของ Factorial

```
int main()
{
    int ans;
    ans = 6;
    cout << ans;
}
```

```
int fact(int x) //3
{
    if(x <= 1)
        return 1;
    else
        return x * 2;
}
```

```
int fact(int x) //2
{
    if(x <= 1)
        return 1;
    else
        return x * 1;
}
```

```
int fact(int x) //1
{
    if(x <= 1)
        return 1;
    else
        return x * fact(x-1);
}
```





Example 3 Bad recursive

```
0 int bad(int n)
1 { if (n==0)
2     return 0;
3     else
4     return bad(n/3 + 1+ n - 1);
5 }
```

Example 4 Printout

```
0 void printout(int n)
1 { if( n >=10 )
2     printout(n/10);
3     cout << n%10;
4 }
```



Example 5 Print again!

```
0 void printout(int n)
1 { if( n >=10 )
2   {
3       cout << n%10;
4       printout(n/10);
5   }
```



การบ้าน 1-1

1. จงเขียนโปรแกรมหาค่า Fibonacci กำหนด function Fibonacci ดังนี้
 $F_0=0, F_1=1$ and $F_n = F_{n-1} + F_{n-2}$ for $n>1$

แสดงผลลัพธ์เป็นค่า fibonacci ตั้งแต่ 0-19

$$F(0) = 1$$

$$F(1) = 1$$

$$F(2) = 1$$

...

$$F(19) = 4181$$



การบ้าน 1-2

2. จากตัวอย่าง printout จงเขียนโปรแกรมรับตัวเลขจำนวนเต็ม 1 ค่า พิมพ์เลขจากหลังไปหน้า และหน้าไปหลัง ด้วยการใช้วิธีการ recursive แสดงผลลัพธ์ 4 ค่า ดังตัวอย่าง

• Input	ตย. 123456	<u>ผลลัพธ์</u>
• Back to front	ตย. 654321	Input : 123456
• Back to front cut last	ตย. 54321	Output :
• Front to back	ตย. 123456	1) 654321
• Front to back cut last	ตย. 12345	2) 54321
		3) 123456
		4) 12345



1.2 Algorithm Analysis

Definition : An algorithm is a clearly specified set of simple instructions to be followed to solve a problem.

- correct
- time or space



Example 6 Running time Calculations

```
int sum(int n)
{
    int partialSum;
    partialSum=0;
    for(int i=1; i<=n; i++)
        partialSum += i*i*i;
    return partialSum;
}
```



1.3 Mathematics Reviews

1) Exponents

$$X^A X^B = X^{A+B}$$

$$\frac{X^A}{X^B} = X^{A-B}$$

$$(X^A)^B = X^{AB}$$

$$X^N + X^N = 2X^N \neq X^{2N}$$

$$2^N + 2^N = 2^{N+1}$$

2) Logarithms

$$X^A = B \rightarrow \log_x B = A$$

$$\log_A B = \frac{\log_c A}{\log_c B}$$



3) Series

$$1 + 2 + 3 + \dots + N$$

$$\sum_{i=1}^N i = \frac{N(N+1)}{2}$$

$$1^2 + 2^2 + 3^2 + \dots + N^2$$

$$\sum_{i=1}^N i^2 = \frac{N(N+1)(2N+1)}{6}$$

คำถาม

$$1 + 2 + 3 + \dots + (N-1) = ?$$



1.4 Mathematical Background

Definition : $T(N)=O(f(N))$ if there are positive constants c and n_0 such that $T(N) \leq cf(N)$ when $N \geq n_0$.

- Give two functions
- we compare their relative rates of growth.

Although $1000N$ is larger than N^2 for small value of N , N^2 grow at a faster rate, and thus N^2 will eventually be the larger function.



Definition : $T(N)=O(f(N))$ if there are positive constants c and n_0 such that $T(N) \leq cf(N)$ when $N \geq n_0$.

$T(N) = 3N+5$ BigO คือ $O(N)$ หรือไม่?

จากนิยาม หา $3N + 5 \leq c \cdot N$ เลือก $c = 4$ และ $n_0 = 5$

แทนค่า $N = 5$ implies $3(5) + 5 \leq 4(5) = 20$ (จริง)

แทนค่า $N = 6$ implies $3(6) + 5 \leq 4(6) = 24$ (จริง)

เนื่องจากเราสามารถหา $c=4$ และ $n_0=5$ ที่ทำให้เงื่อนไขนี้เป็นจริงไปตลอดได้ จึงสรุปตามนิยามได้ว่า $3N + 5 = O(N)$



Definition : $T(N)=O(f(N))$ if there are positive constants c and n_0 such that $T(N) \leq cf(N)$ when $N \geq n_0$.

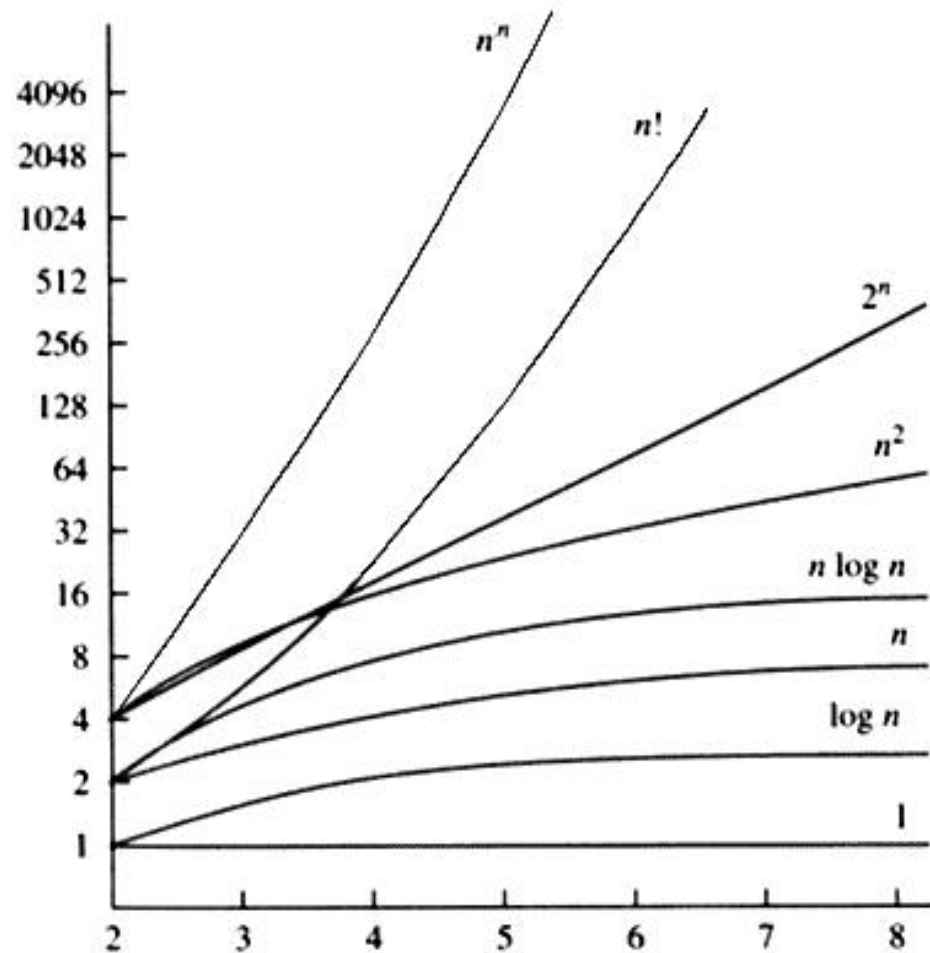
- $T(N) = 1000N$
- $f(N) = N^2$
- เมื่อ $N=1$, $c=1$ จะเห็นว่า $1000N > N^2$
- เมื่อ N มีค่ามากขึ้น จนถึง 1000 $1000N = N^2$
- เมื่อ N มากกว่า 1000 $1000N < N^2$

We can say that $1000N = O(N^2)$

$1000N$ เป็นฟังก์ชันที่โตไม่เร็วกว่า N^2



Function	Name
C	Constant
$\log N$	Logarithmic
$\log^2 N$	Log-squared
N	Linear
NlogN	
N^2	Quadratic
N^3	Cubic
2^N	Exponential





1.5 General Rules

Rule 1– For loop

The running time of a for loop is at most the running time of the statements inside the for loop(including tests) times the number of iterations.

```
for(i=0; i<n; i++)  
    k++;
```

Rule 2– Nested loops

Analyze these inside out. The total running time of a statement inside a group of nested loop is the running time of the statement multiplied by the product of the sizes of all the loops.

```
for(i=0; i<n; i++)  
    for(j=0; j<n; j++)  
        k++;
```



Rule 3– Consecutive statement For loop

Add.

```
for(i=0; i<n; i++)  
    a[i]=0;  
for(i=0; i<n; i++)  
    for(j=0; j<n; j++)  
        a[i]+=a[j]+i+j;
```



Rule 4– If/Else

The running time of an if/else statement is never more than the running time of the test plus the running times of S1 and S2

```
if (condition)
    S1
else
    S2
```



คำถาม จาก code ด้านล่าง จงหาค่า bigO

```
1. int example(int numbers[], int size)
{
    int total = 0;
    for (int i = 0; i < size; i++)
    {
        total += numbers[i];
    }
    return total;
}
```



คำถาม จาก code ด้านล่าง จงหาค่า bigO

```
2. int example(int matrix[][3], int rows, int cols) {  
    int total = 0;  
    for (int i = 0; i < rows; i++) {  
        for (int j = 0; j < cols; j++) {  
            total += matrix[i][j];  
        }  
    }  
    return total;  
}
```



คำถาม จาก code ด้านล่าง จงหาค่า bigO

```
3. int example(int numbers[], int size)
{   if (size > 0)
        return numbers[0];
    else
        return -1;
}
```



Example 7 Maximum subsequence sum

4 -3 5 -2 -1 2 6 -2

4

-3

4 + -3

-3 + 5

4 + -3 + 5

-3 + 5 + -2

4 + -3 + 5 + -2

-3 + 5 + -2 + -1

...

...

4 + -3 + 5 + -2 + -1 + 2 + 6 + -2

-3 + 5 + -2 + -1 + 2 + 6 + -2



Example 7(ต่อ) Maximum subsequence sum

4 -3 5 -2 -1 2 6 -2

```
int MasSubsequenceSum(int a[], int N)
{
    int ThisSum, MaxSum, j;
    for(j=0; j<N; j++)
    {
        ThisSum += A[j];
        if( ThisSum > MaxSum)
            MaxSum = ThisSum;
        else if( ThisSum < 0 )
            ThisSum=0;
    }
    return MaxSum;
}
```